The Foundation
0%

Module 1: QUICK START (...
1.1 What You Need (5 Min Set...
1.2 Your First Folder
1.3 How to Structure Any Pro...

Module 2: WHY THIS WO...
2.1 Video Text Guide (Series ...
2.2 One Line of Python Trigge...
2.3 How a 1953 Word Game E...
2.4 The Ladder That Explains ...
2.5 Clawdbot (Moltbot) Has 1...
2.6 Video as Code: My AI Ani...
2.7 From Nazi Psychology to ...

Module 3: FOLDER ARCHI...

2

3.2 Customizing for Your Use ...
3.3 Common Mistakes and H...

4.4 Making Claude Understand Your Project

What You'll Get From This

You will write a CLAUDE.md file that makes Claude Code behave like it has been on your project for a month. One file. Ten minutes. The difference is immediate.

Video: Session 4 — Making Claude Code Understand Your Project

> **Coming Soon**

The Lesson

One file changes everything

In Session 1 you installed Claude Code. In Session 2 you used it on real tasks. In Session 3 you used Desktop for thinking. All of that worked. But there is a problem you have probably already noticed: Claude Code does not know your project.

It does not know your conventions. It does not know your file structure. It does not know which libraries you use, which

would already know.

The fix is one file: **CLAUDE.md**.

Module 4: CLAUDE CODE ...

4.1: Install and First Use

4.2: Claude Code in Practice

4.3: Claude Desktop as a Thin...

4.4 Making Claude Understan...

4.5: Where This Goes

SECTION 5: WHERE TO G...

What a CLAUDE.md is

A markdown file that sits in the root of your project folder. Claude Code reads it automatically every time it starts. It tells Claude what this project is, how it works, and what matters. Think of it as an onboarding document for a new hire. Except the new hire reads the entire thing in two seconds and follows every word.

Before vs after

Run the same task on the same project. First without a CLAUDE.md. Then with one. The difference:

Without CLAUDE.mdWith CLAUDE.mdGeneric code styleMatches your conventionsGuesses at file structureKnows where things liveDoes not know your commandsRuns your actual test suiteSuggests wrong librariesUses your dependencies

This is the same principle from **Section 1.2**: structured context changes the output. The CLAUDE.md is a more focused, project-specific version of the identity and context files you set up in Section 1. If you skipped Section 1 and jumped straight to Section 4, this lesson still works. But go back to 1.2 afterward to understand the bigger picture.

> [📌 JAKE: The before/after demo is the most important visual in this lesson. Same task, same project. Show the generic output without CLAUDE.md, then the specific output with it. Side by side if possible.]

What goes in it

1. Project overview — Two to three sentences. What is this? What does it do?

2. Tech stack — What languages, frameworks, databases, tools.

3. How to run things — Dev server, tests, build commands. The commands Claude needs to know.

4. Key conventions — Naming patterns, file structure, architectural patterns you follow.

5. What to avoid — Things Claude should not do. Libraries you do not use. Patterns you have moved away from.

Example CLAUDE.md

```
# My Web App

React 18 + Express + PostgreSQL + TypeScript

## Commands
npm run dev | npm run api | npm test

## Conventions
Functional components only. Routes in src/api/.
All database queries go through src/db/queries/.

## Avoid
No class components. Don't modify db/migrations directly.
Don't use Moment.js (we use date-fns).
```

That is it. Fifteen lines. Ten minutes. The difference in Claude's behavior is immediate.

Tips

Write it for a smart person who just joined your project. That is literally what Claude is. If a new team member would need to know it on day one, put it in the CLAUDE.md.

living document. When you add a new convention, change a library, or shift your architecture, update the file. This is the same advice from **3.3 (Common Mistakes)**: stale context files are the most common reason Claude starts "getting worse."

Do not overthink it. A mediocre CLAUDE.md beats no CLAUDE.md every time. You can always improve it later. The first version takes 10 minutes. Each edit takes 30 seconds. Start now.

This is not just for code projects. If your project is a folder full of documents, the CLAUDE.md describes what those documents are, how they are organized, and what you are using them for. The same principle applies to any structured folder, whether it contains code, writing, research, or client files.

Write your CLAUDE.md

Do this now.

1. Pick a project. Work project, side project, or even a document folder.
2. Create a file called `CLAUDE.md` in the root of the folder.
3. Fill it in using the template:

```
# [Project Name]

## Overview
[2-3 sentences. What is this? What does it do?]

## Tech Stack
[Languages, frameworks, tools. Or "document types" if not a

## Commands
[Dev, test, build. Or "how to use these files" if not code.]

## Conventions
[Patterns, naming, structure.]
```

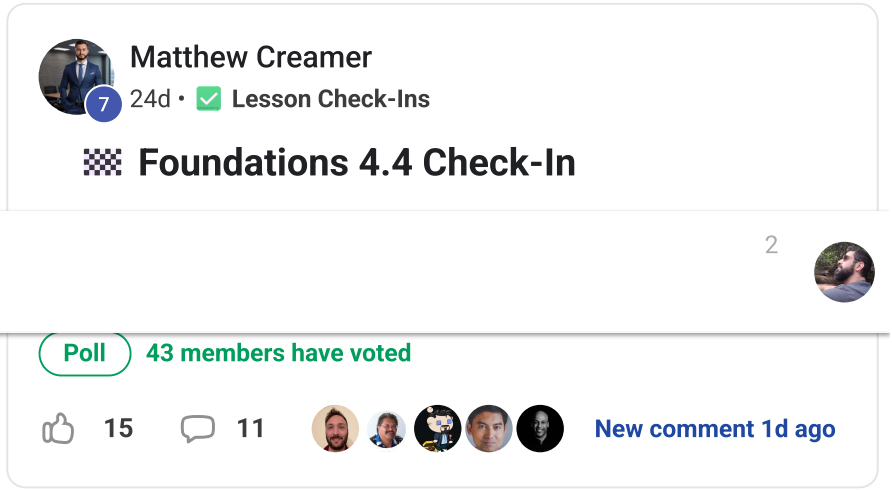

4. Save it. Open Claude Code in the folder. Run a task.

5. (Optional) Temporarily move the CLAUDE.md out of the folder. Run the same task. Compare the outputs. Move it back.

The comparison is what makes the value click. You will not go back.

Go Deeper

- The CLAUDE.md is a project-specific version of the identity file from **1.2: Your First Folder**. If you want to understand the full three-layer architecture (identity → workspace context → skills), go to **Section 3**.
- Common mistakes with context files (too long, too focused on personality, never updated) are covered in **3.3: Common Mistakes and How to Fix Them**. Every one of those mistakes applies to CLAUDE.md files too.
- If you have multiple projects or different types of work, **Session 4.5** shows how to scale this with workspace routing.

Next Up: 4.5 Where This Goes

You have one CLAUDE.md for one project. What happens when you have ten projects? Or a complex workflow with different types of tasks? Session 5 shows the full architecture and where all of this scales.